ENGINEERING BLUEPRINT / 01

CIT CAMPUS SCHEDULING MVP

What to build. What to use. Where to learn.

PURPOSE

A practical build guide for the first working milestone: load the official CIT dataset, generate a weekly teaching timetable with a constraint solver, validate it independently, and export schedule.json.

27 ROOMS	40 FACULTY	14 GROUPS	189 SESSIONS
62 courses	243 teaching hours	25 booking requests	41 midsem exams

Recommended MVP stack

Backend	Scheduling	Frontend	Data / validation
Python + FastAPI	Google OR-Tools CP-SAT	Single HTML + CSS + JavaScript	cit_public.json + official validator.py

Prepared from the CIT problem statement and Crunch Week starter pack. External links are included as learning references, not as code to clone.

Document version: 31 October 2026 | Scope: MVP 1 - weekly teaching timetable

SCOPE AND SUCCESS CRITERIA

01. Define the MVP before expanding it

The first MVP should prove the hardest core capability: generating a valid weekly teaching timetable from the supplied dataset. It is not the complete hackathon submission. Keep the first build narrow so solver correctness can be tested before adding bookings, exams, AI, authentication or a large dashboard.

> **MVP 1 ACCEPTANCE TARGET**
>
> Read the real cit_public.json; generate one placement for every one of the 189 teaching sessions; satisfy all teaching hard constraints; independently validate the result; and export the generated assignments. The complete official export will still require exams and Spectra booking decisions.

Dataset and scheduling facts

Item	CIT value	Why it matters
Rooms / faculty / groups	27 / 40 / 14	Resource and cohort conflict domains
Courses / sessions	62 / 189	Schedule sessions, not course totals
Weekly teaching load	243 hours	Duration-aware placement is required
Spectra requests	25 (1 cancelled)	Later booking-engine phase
Midsem exams	41	Separate exam-week model required
Teaching calendar	Mon-Fri, P1-P9	One-hour periods; lunch after P5
Event period range	P10-P13	Evening event windows; not normal classes

MVP 1 versus full product

Capability	MVP 1 now	Later full platform
Dataset inspection	Yes	Admin uploads / validates changed files
Teaching timetable	Yes	Manual edits + group/faculty/room views
Midsem exam scheduling	No	Dedicated exam CP-SAT model
Spectra bookings	No	NL request parsing, priority, approval, reason
Disruption repair	No	Minimum-change solver + Admin approval
Roles / database / audit	No	RBAC, persistent state, audit trail
AI assistant	No	LLM parses / explains; code validates

Source of truth: the problem statement and starter-pack README define the requirements and JSON contract. The supplied validator.py defines scoring and machine-checkable outcomes. Do not replace those rules with assumptions from reference projects.

KEEP THE FIRST VERSION SIMPLE

02. Technology choices and architecture

Layer	Use in MVP 1	Reason / role
Language	Python 3.11+	Dataset parsing, constraint model and APIs
Backend	FastAPI + Uvicorn	HTTP API, serves the HTML page, exposes /docs
Optimization	Google OR-Tools CP-SAT	Boolean / interval decisions, hard constraints, objective
Validation	Pydantic + explicit Python checks	Validate API inputs and independently check assignments
Frontend	One index.html with HTML/CSS/vanilla JS	No Node build step; quick inspect/generate/view workflow
Data source	Official cit_public.json	Do not hard-code rooms, faculty or sessions
Persistence	JSON export for MVP 1	Add PostgreSQL when users, approvals and versions exist

Request-to-schedule flow

Step	Component	Action
1	Browser UI	Admin clicks Generate Timetable and sends solver options
2	FastAPI endpoint	Loads cached CIT JSON and calls the solver function
3	Candidate builder	Filters individual placements that violate eligibility rules
4	CP-SAT model	Selects exactly one placement per session under global constraints
5	Internal validator	Checks hard rules independently of the solver model
6	Serializer / export	Writes assignments to exports/schedule.json
7	Browser UI	Displays solver status, penalty, validation and timetable grid

DESIGN PRINCIPLE

The solver is a pure scheduling component: it receives structured dataset objects and returns assignments plus solver metadata. It should not depend on HTTP, browser code, user sessions or database connections. FastAPI orchestrates; OR-Tools decides placements.

Do we need a database now?

Not for MVP 1. The supplied JSON is the scheduling source of truth and the app only needs to export a generated timetable. Add PostgreSQL later for user accounts, approval workflows, booking history, schedule versions, audit logs and durable state. Avoid introducing migrations and ORM complexity before the solver is proven.

PYTHON + FASTAPI

03. Backend: what to implement

The current prototype deliberately keeps the backend in app.py. Once the model stabilizes, split the code into the modules below. Do not create an empty enterprise folder structure before the first endpoint and dataset load work.

Module / file	Responsibility	Build order
app.py / main.py	FastAPI app, CORS, endpoint registration, error mapping	1
services/dataset_service.py	Read JSON, check required keys, cache dataset	2
schemas/	Pydantic request / response contracts	3
scheduler/context.py	Build ID indexes and availability lookup sets	4
scheduler/time_model.py	Period, duration, lunch and occupied-period helpers	5
scheduler/candidates.py	Create valid (day, start, room) domains	6
scheduler/model_builder.py	Variables, hard constraints and objective	7
scheduler/solve.py	Run CP-SAT and map solver status	8
services/validation_service.py	Independent checks and validator invocation	9
services/export_service.py	Serialize the official schedule.json structure	10

MVP 1 API contract

Method + route	Purpose
GET /api/health	Confirm the service is alive
GET /api/dataset/summary	Show dataset ID, counts, days and teaching periods
GET /api/metadata	Return room, faculty and student-group filters
GET /api/rooms	Return rooms and their properties
GET /api/sessions	Return the 189 session records
POST /api/timetable/generate	Run CP-SAT and validate the generated schedule
GET /api/timetable/latest	Return latest assignments and solver metrics
POST /api/timetable/validate	Re-run internal teaching-timetable validation
GET /api/export/schedule	Download the generated schedule.json

Generate request

```
POST /api/timetable/generate
Content-Type: application/json

{
  "time_limit_seconds": 90,
  "num_workers": 4
}
```

Solver status handling: OPTIMAL = feasible and proven optimal for the modeled objective; FEASIBLE = valid solution found but optimality not proven; INFEASIBLE = model has no solution; UNKNOWN = no conclusion within the limits; MODEL_INVALID = fix the model. Never export an invalid or unvalidated assignment.

OR-TOOLS CP-SAT

04. Scheduling engine: build in this order

The engine assigns each session one day, one starting period and one room. Faculty and student groups are already assigned in the starter data. The engine therefore chooses only the placement variables; it does not redesign course enrolments.

Stage	Input	Output / check
1. Load context	calendar, rooms, faculty, groups, sessions	ID maps, days, teaching periods, availability sets
2. Candidate generation	one session + all resource data	Allowed (day, start_period, room) triples
3. Domain diagnostics	candidate domains	Report sessions with zero eligible placements
4. Decision variables	candidate placements	Boolean / integer variables for chosen placement
5. Hard constraints	resources + durations	No overlaps; one assignment; all CIT hard rules
6. Objective	S1-S6 preferences	Minimize total soft penalty
7. Solve	CP-SAT model + limits	OPTIMAL / FEASIBLE / no-solution status
8. Verify + export	selected placements	Independent validation, then schedule.json

Candidate generation comes before conflict solving

For each session, enumerate the allowed teaching days, start periods and rooms. Exclude candidates that fail individual eligibility: incompatible room type, disallowed room use, insufficient capacity, missing accessibility, room maintenance, faculty unavailability, lunch crossing, invalid lab start or fixed-slot mismatch. Then the solver handles conflicts between different sessions.

> **WHY THIS SEPARATION MATTERS**
>
> Eligibility is local to one session placement. Clashes are global because they depend on what other sessions choose. Keeping these checks separate makes the model smaller, easier to explain and easier to debug.

Decision model

For a candidate placement j of session s, define x(s,j) as a Boolean: 1 means select that placement, 0 means do not select it. Every session must select exactly one candidate:

```
For every session s:  SUM over its candidates j of x(s,j) = 1
For every pair of conflicting placements a,b:  x(a) + x(b) <= 1
```

For resource conflicts, use duration-aware intervals and NoOverlap constraints where practical. Pairwise incompatibility constraints are also valid for a small candidate model. Be careful: a two-hour session starting at P2 occupies both P2 and P3; checking only start periods is incorrect.

DO NOT WEAKEN THE OFFICIAL RULES

05. CIT constraints to encode

Hard constraints - all must hold

Rule family	Implementation requirement
Exactly once	Every one of the 189 session IDs appears exactly once in the teaching assignment set.
Faculty no-overlap	A faculty member cannot be teaching overlapping sessions on the same day.
Student-group no-overlap	A group cannot attend overlapping sessions. Combined lectures count for every listed group.
Room no-overlap	A room cannot host two overlapping sessions.
Capacity and room use	Room capacity >= session size; room type and allowed_uses must both match.
Availability	Respect faculty unavailability and room maintenance periods.
Duration / lunch	A duration-d session occupies p through p+d-1 and cannot cross the P5/P6 lunch boundary.
Labs	Three-hour labs start only at P2 or P6.
Accessibility	If session or relevant group requires accessibility, select an accessible room.
Elective basket	Same basket_slot sessions share day and start period, use separate rooms, and count once for group occupancy.
Faculty daily load	Do not exceed each faculty member max_hours_per_day (4 hours in CIT data).
Fixed slot	Preserve the exact fixed day and start_period.

Soft constraints - minimize penalties

Score	Penalty meaning from validator.py	Penalty
S1	Faculty avoid preference: each avoided teaching hour	2 points / hour
S2	Free teaching periods between a group's first and last occupied period in a day	1 point / gap
S3	Group daily teaching load above 6 hours	3 points / excess hour
S4	Extra same-course lecture on a group/day; basket course lectures also counted by course/day	3 points / extra lecture
S5	Room utilization below 40% (session size / room capacity)	1 point / session
S6	Session occupies P9	1 point / session

IMPLEMENTATION RULE

Hard constraints define feasibility. Soft constraints define preference. Never turn a soft preference (such as avoiding P9) into a hard prohibition. Reproduce the official validator.py definitions exactly; it is the scoring authority.

SINGLE-PAGE PROTOTYPE

06. Frontend: what the HTML/JavaScript MVP should show

For the first milestone, one index.html is enough. Use CSS Grid and a small amount of vanilla JavaScript. The page should be an operations console, not a full admin portal.

UI area	Minimum behavior
Dataset summary cards	Rooms, faculty, groups, courses, sessions, exams and booking-request counts
Generate controls	Time limit and worker count; show progress / disabled state while solving
Solver result panel	OPTIMAL / FEASIBLE status, solve time, candidate count, objective and best bound
Validation panel	Hard-violation count, rule code and diagnostic detail
Timetable grid	Weekday columns or rows; periods P1-P9; color by lecture/tutorial/lab
Filters	Group, faculty, room and day; filter existing assignments client-side
Export action	Download schedule.json from backend endpoint

Current MVP file map

File / folder	Role
app.py	FastAPI app, dataset loader/cache, candidate builder, CP-SAT solver, internal validator, soft penalties and endpoints
index.html	Single HTML/CSS/JS dashboard calling the API
data/cit_public.json	Original supplied dataset
validator.py	Official supplied checker - keep unchanged
exports/schedule.json	Generated teaching timetable output
examples/	Starter request and disruption examples for later phases
requirements.txt	FastAPI, Uvicorn and OR-Tools dependencies
AI_USAGE.md / THIRD_PARTY.md	Required declarations; keep updated

UI references and choices

- For MVP 1: use a custom grid because CIT has a fixed nine-period structure and fixed day columns.
- For a later React/Next.js interface: study shadcn/ui Sidebar, Table, Card, Badge and Chart components.
- FullCalendar is an option for a broader calendar view. Its standard React component is MIT-licensed; its Premium Scheduler/resource-timeline features have separate licensing. Check the current license before choosing a resource timeline.

WINDOWS POWERSHELL WORKFLOW

07. Run locally and prove the milestone

Use the included MVP ZIP as the starting point. Extract the folder, open it in VS Code, and run commands from the folder containing app.py and requirements.txt.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
uvicorn app:app --reload

# Open in browser:
http://127.0.0.1:8000
# API explorer:
http://127.0.0.1:8000/docs
```

Recommended test sequence

Test	Expected outcome
Health endpoint	GET /api/health returns healthy
Dataset summary	Counts match the starter pack: 27 rooms, 40 faculty, 14 groups, 62 courses, 189 sessions, 25 booking requests, 41 exams
Candidate domains	Every session has at least one candidate; otherwise inspect per-filter diagnostics
Generate timetable	Solver returns OPTIMAL or FEASIBLE, not UNKNOWN / INFEASIBLE / MODEL_INVALID
Internal validation	Teaching timetable has zero hard violations
Soft score	S1-S6 recomputed by validator logic; compare model objective and score
Export	schedule.json contains every teaching session exactly once

Run the official checker

```
python validator.py --data data/cit_public.json \
  --schedule exports/schedule.json

# JSON report:
python validator.py --data data/cit_public.json \
  --schedule exports/schedule.json --json report.json
```

IMPORTANT SCOPE LIMIT

MVP 1 exports the sessions array only. It does not yet create fest_week.bookings, session_overrides or the 41 exam assignments. Therefore, the full official validator is expected to report missing exams / booking decisions until those later modules are implemented. Use the internal teaching validation to assess MVP 1.

Do not hand-edit schedule.json. Every exported assignment must be produced by the application. Keep the original validator.py unchanged.

ADD ONE SUBSYSTEM AT A TIME

08. Roadmap from MVP 1 to the full submission

Phase	Build next	Reuse from MVP 1
1. Teaching timetable	Complete and validate all 189 sessions	Dataset loader, context, time model, CP-SAT patterns, export
2. Exams	Separate exam-week model; AM/PM slots; half room capacity; multi-room exams	Room metadata, interval/conflict checks, validator integration
3. Booking engine	Parse 25 requests, cancellations, role category caps, priority and alternatives	Room data, calendar, interval conflicts, validation and reasons
4. Week 7 overrides	Handle approved institute bookings displacing sessions only with a valid moved session	Timetable solver and schedule serialization
5. Disruption repair	Apply changes, minimize moved sessions, compare before/after, Admin approval	CP-SAT model, baseline assignment, change-cost objective
6. Roles and persistence	Admin, coordinator, faculty, club; cannot approve own request; audit log	FastAPI routes and schemas; add PostgreSQL / SQLAlchemy
7. AI copilot	English/Hinglish extraction and explanations; manual fallback always works	Pydantic schemas, booking APIs and validation gate
8. Full UX and release	Group/faculty/room views, approvals, conflicts, room utilization, demo accounts	Existing API contracts; upgrade UI as needed

Critical architectural rule for AI

LLM IS NOT THE SCHEDULER

An LLM may extract intent, ask clarifying questions, fill a structured request and explain a result. Python rules and OR-Tools must decide whether a booking or schedule is valid. Validate every proposal in code before any change is saved. If the LLM API fails, the manual form must still work.

Rescheduling objective

For a disruption, start from the current valid timetable and apply the disruption to the data. Solve again under all hard constraints, while penalizing changes from the baseline. Prefer a lexicographic objective: (1) zero hard violations, (2) minimize sessions whose day/start changes, (3) minimize room-only changes, then (4) improve soft penalties. Save as a proposal and apply only after Admin approval.

USE REFERENCES FOR PATTERNS, NOT AS A PROJECT TO CLONE

09. Reference library: where to learn and what to take

The CIT problem statement, starter-pack README, sample export and official validator are authoritative for this project. The external resources below help with product workflows, solver modeling, API structure and presentation. Confirm package versions and licenses before adding dependencies.

R1 UniTime - product workflow and university scheduling Course timetabling, examination timetabling, instructor scheduling, event management and schedule-change workflows. Study screens and documentation; do not copy its large Java implementation.
LINK: https://help.unitime.org/documentation

R2 UniTime - online demo Explore how a university scheduling product organizes course, exam and event functions.
LINK: https://demo.unitime.org/

R3 UniTime - source repository Reference for large-scale product structure only; not a suitable code base to transplant into this Python MVP.
LINK: https://github.com/UniTime/unitime

R4 FU School Scheduler - closest stack reference FastAPI + OR-Tools + Next.js example. Study API contracts and separation of backend / solver. Its README states a smaller constraint scope; it does not cover CIT room allocation, CIT soft scores or the full problem.
LINK: https://github.com/OsmanAltunbag/fu-school-scheduler

R5 Google OR-Tools - official documentation Primary learning resource for CP-SAT variables, constraints, scheduling and optimization.
LINK: https://developers.google.com/optimization

R6 OR-Tools - scheduling recipes Study interval variables, optional intervals, NoOverlap, cumulative constraints and breaks in calendars.
LINK: https://github.com/google/or-tools/blob/stable/ortools/sat/docs/scheduling.md

R7 OR-Tools - employee scheduling example A smaller CP-SAT example to understand Boolean decisions, at-most-one constraints and schedule feasibility.
LINK: https://developers.google.com/optimization/scheduling/employee_scheduling

R8 Timefold school timetabling Good explanation of room, teacher and student-group hard conflicts and soft preferences. The quickstart is Java/Quarkus; reuse concepts, not the stack.
LINK: https://github.com/TimefoldAI/timefold-quickstarts/tree/stable/use-cases/school-timetabling

R9 FastAPI tutorial Learn request models, routing, dependency injection, response handling and interactive API docs.
LINK: https://fastapi.tiangolo.com/tutorial/

R10 Pydantic models Learn typed request / response objects and validation of structured data.
LINK: https://docs.pydantic.dev/latest/concepts/models/

R11 shadcn/ui components For a later React/Next.js dashboard: Sidebar, Table, Card, Badge, Dialog and Chart components.
LINK: https://ui.shadcn.com/docs/components

R12 FullCalendar React Optional React calendar integration. Standard package is MIT-licensed; resource timeline / Scheduler features may require Premium licensing.
LINK: https://fullcalendar.io/docs/react

R13 FullCalendar licensing Check standard versus Premium terms before using Scheduler/resource timeline in a submission.
LINK: https://fullcalendar.io/license

PRACTICAL STUDY ORDER

1) FU School Scheduler for a small FastAPI/solver pattern. 2) OR-Tools scheduling recipes to implement the model correctly. 3) UniTime demo/docs for the breadth of university workflows. 4) Timefold for constraint-design ideas. 5) shadcn / FullCalendar only when the interface grows beyond the single-file MVP.

Source notes: CIT-specific counts, rules and export fields are from the supplied problem statement and Crunch Week starter-pack README. S1-S6 meanings and point values above were cross-checked against the supplied validator.py. External link descriptions are based on the linked project documentation as reviewed on 31 October 2026.